COURS · IA & OPENCODE

Comprendre l'IA et OpenCode

20 schémas pour ne plus jamais être perdu

Une fiche par thématique : l'essentiel en une phrase et deux schémas pour comprendre d'un coup d'œil.

- DÉCOUVRIR
- INSTALLER
- PROVIDERS & MODÈLES
- LES AGENTS
- ÉTENDRE OPENCODE
- PRATIQUE & MÉTHODE

DÉCOUVRIR · INSTALLER · CONFIGURER · ÉTENDRE · MAÎTRISER

SOMMAIRE

20 FICHES · 2 SCHÉMAS CHACUNE

Ce que tu vas comprendre

Découvrir IA & OpenCode

01 Comprendre l'IA sans jargon

02 Le vocabulaire : modèle, provider, agent

Installer OpenCode

03 Installer OpenCode (toutes les méthodes)

04 Premier lancement & l'interface

Providers & modèles

05 Qu'est-ce qu'un provider ?

06 Trouver des modèles gratuits

07 Choisir son modèle (gratuit ou payant)

08 Configurer ses providers (fichier de config)

09 Modèles locaux (Ollama, LM Studio, vLLM)

10 Réduire les coûts (et rester efficace)

Les agents

11 Qu'est-ce qu'un agent ?

12 Créer un agent local (pas à pas)

13 Trouver & intégrer des agents

Étendre OpenCode

14 Les Skills : des compétences réutilisables

15 Les plugins : brancher des outils

16 Les commandes : tes raccourcis

17 Les serveurs MCP : connecter le monde

Pratique & méthode

18 Bien prompter & le fichier AGENTS.md

19 Mini-projet : ton assistant de A à Z

20 Synthèse, dépannage & ressources

Comprendre l'IA sans jargon

Les premières pendules molles caressées par l'aube.

Un LLM prédit du texte : très fort pour transformer, fragile sur les faits.

◆ COMMENT IL RÉPOND

Ta demande
le prompt

→

Modèle (LLM)
prédit la suite

→

Réponse probable
pas une vérité

Il ne cherche pas la vérité : il prédit le texte le plus probable.

◆ CE QU'IL FAIT BIEN / MAL

● Fort

- Transformer
- Expliquer
- Résumer

● Fragile

- Faits précis
- Chiffres
- Actualité

Le vocabulaire : modèle, provider, agent

Un alphabet doré qui apprend à respirer.

> **Modèle = moteur · Provider = robinet · Token = coût · Contexte = mémoire.**

◆ L'ANALOGIE DE L'EAU

Modèle	le moteur
Provider	le robinet
API + clé	l'accès
Agent	le moteur + une mission

◆ DEUX MOTS DE COÛT

Token

- Unité de facturation
- ≈ un morceau de mot

Contexte

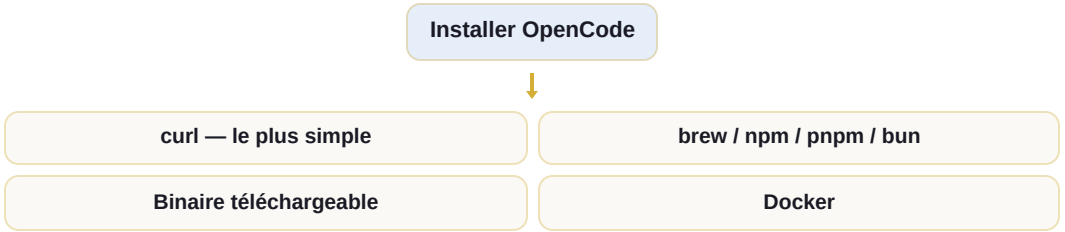
- Mémoire de travail
- Se remplit vite

Installer OpenCode (toutes les méthodes)

Poser la première pierre d'une cité de lumière.

> **Une commande suffit pour installer ; on vérifie toujours la version.**

◆ QUATRE MÉTHODES



◆ VÉRIFIER / DÉSINSTALLER

- **On vérifie**
  - opencode --version

- **Désinstaller**
  - Le binaire part
  - La config reste
  - Les sessions restent

Premier lancement & l'interface

Le premier souffle éveille la machine endormie.

On lance OpenCode dans le dossier du projet, puis /connect et /models.

◆ LE BON RÉFLEXE

cd mon-projet
se placer dedans

→

opencode .
démarrer ici

→

L'interface s'ouvre
prête à discuter

Deux commandes pour commencer : /connect puis /models.

◆ TROIS FAÇONS DE LANCER

- **opencode**
  - L'interface (TUI)
  - Le quotidien
- **opencode run**
  - Un script en une ligne
- **opencode mini**
  - Version légère

Qu'est-ce qu'un provider ?

Choisir la source d'où jaillit l'étincelle.

Un provider fournit des modèles par API ; un ou deux suffisent.

◆ QUI PARLE À QUI



◆ BONNE PRATIQUE

Commencer
- 1 ou 2 providers

Ensuite
- Plusieurs = résilience

Jamais
- Partager sa clé

La clé est rangée localement par OpenCode, pas en clair dans un fichier.

Trouver des modèles gratuits

Les coffres ouverts du soleil, sans péage.

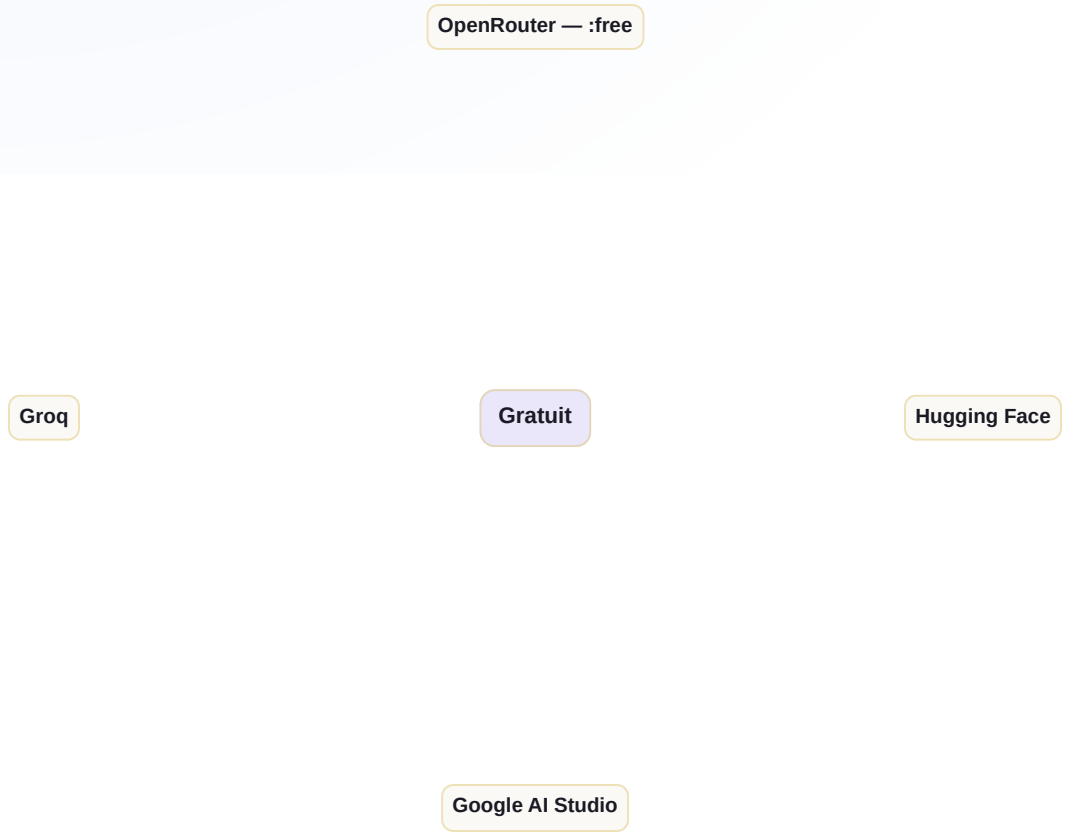
> **Le gratuit existe, mais avec des quotas serrés.**

◆ QUOTAS OPENROUTER GRATUIT



Les modèles gratuits portent le suffixe :free.

◆ OÙ TROUVER DU GRATUIT



Choisir son modèle (gratuit ou payant)

Trier les étoiles selon leur juste éclat.

Cinq critères, dont un non négociable : le support des outils.

◆ CINQ CRITÈRES

Indispensables

- Outils (tool calling)
- Fenêtre de contexte

À arbitrer

- Prix
- Qualité du code
- Vitesse

Sans tool calling, aucun agent ne fonctionne.

◆ ON PAIE PAR MILLION DE TOKENS



Un petit modèle pour le simple, un gros modèle seulement si nécessaire.

Configurer ses providers (fichier de config)

Accorder l'instrument avant le concert.

> **Tout se configure dans un fichier JSONC, secrets exclus.**

◆ DEUX FICHIERS POSSIBLES

Global

- ~/.config/opencode/opencode.jsonc
- Pour toi, partout

Projet

- opencode.jsonc à la racine
- Versionné avec l'équipe

◆ UN SECRET NE S'ÉCRIT JAMAIS EN CLAIR

{env:MA_CLE}
dans le fichier

→

La vraie clé
dans l'environnement

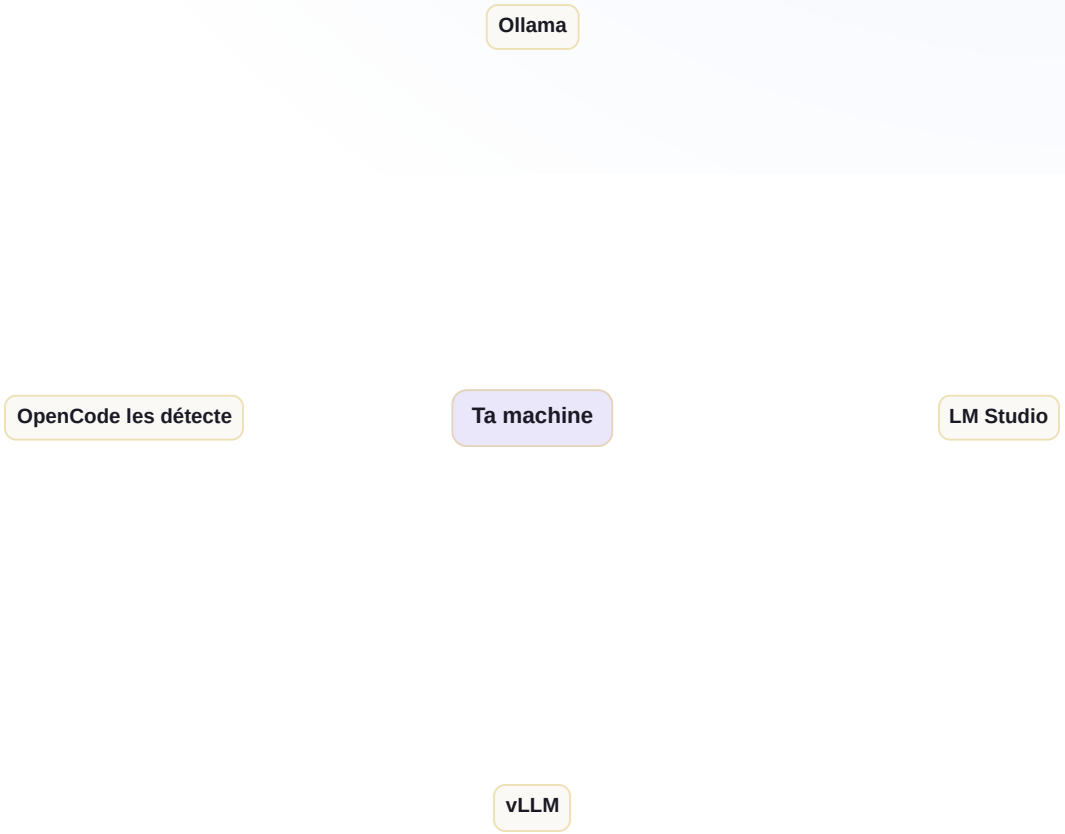
V2 utilise le pluriel : providers, agents, plugins, mcp.servers.

Modèles locaux (Ollama, LM Studio, vLLM)

Une flamme qui brûle sur ta propre table.

> **Local : gratuit et privé, mais limité par ta machine.**

TOUT TOURNE SUR TA MACHINE



LE VRAI COMPROMIS

Avantages

- Hors ligne
- Gratuit
- Privé

Limites

- Lent
- RAM / VRAM
- Outils parfois absents

Vérifie toujours le support des outils avant de choisir un modèle local.

Réduire les coûts (et rester efficace)

Dompter la rivière d'or qui fuit entre les doigts.

> **Le plus gros levier d'économie, c'est le contexte envoyé.**

◆ LES LEVIERS, DU PLUS FORT AU PLUS FAIBLE



◆ LE BON RÉFLEXE

Question simple
petit modèle

→

Grosse tâche
gros modèle

Le fallback évite de rester bloqué quand un quota est atteint.

Qu'est-ce qu'un agent ?

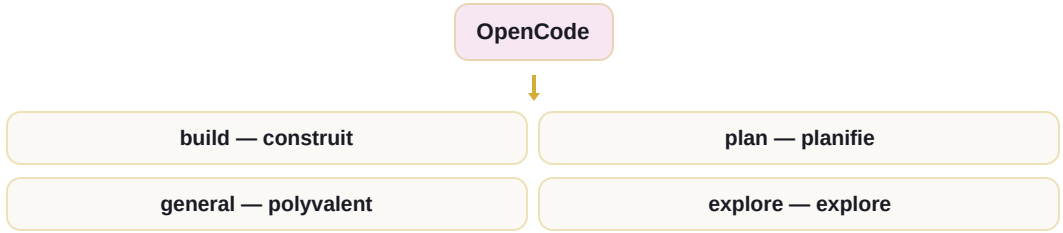
Un automate doré qui rêve d'agir.

> **Un agent = rôle + modèle + outils + permissions.**

◆ LA RECETTE D'UN AGENT

Rôle	une mission claire
Modèle	le moteur
Outils	ce qu'il peut faire
Permissions	ce qu'il a le droit de faire

◆ LES AGENTS INTÉGRÉS



Trois modes : primary, subagent, all. La dernière permission gagne.

Créer un agent local (pas à pas)

Façonner une silhouette de lumière sur mesure.

> **Un agent, c'est un fichier Markdown avec un en-tête.**

◆ ANATOMIE D'UN AGENT

Fichier .md
dans agents/

→

En-tête
description, mode, model

→

Corps Markdown
le prompt système

◆ DEUX EMPLACEMENTS

agents/

↓

~/.config/opencode/agents/

<projet>/.opencode/agents/

default_agent choisit l'agent de départ.

Trouver & intégrer des agents

Fouiller le grand marché des ombres utiles.

> **Un agent peut agir dans ton projet : lis-le avant de l'installer.**

◆ AVANT D'INSTALLER

Trouver
GitHub, communautés

→

Lire
un agent = du code

→

Installer
déposer le .md

→

Adapter
modèle et contexte

◆ POURQUOI LIRE D'ABORD

● Un agent peut

- Lire tes fichiers
- Exécuter des commandes
- Sortir sur le réseau

● Souvent mieux

- Écrire le tien

Les Skills : des compétences réutilisables

Offrir à la machine de nouveaux gestes appris.

> **Une skill n'est chargée que quand elle sert.**

◆ COMMENT ELLE SE CHARGE

Description du dossier
permet de la découvrir

→

@id ou outil skill
chargement à la demande

→

Compétence prête
sans gonfler le contexte

◆ DEUX POINTS D'ATTENTION

L'identifiant

- Vient du chemin, pas du name
- Sensible à la casse

Permission skill

- Autoriser
- Demander
- Bloquer

Les plugins : brancher des outils

Brancher des membres supplémentaires au colosse.

> **Un plugin ajoute des capacités, OpenCode les découvre seul.**

◆ GÉRER UN PLUGIN

plugin add ajouter	→ **list** lister	→ **update** mettre à jour	→ **remove** retirer

◆ OÙ OPENCODE LES CHERCHE

Plugins

↓

.opencode/plugins/

~/.config/opencode/plugins/

Les plugins d'interface se règlent à part, dans cli.json.

Les commandes : tes raccourcis

Murmurer des ordres brefs à l'oreille du sphinx.

> **Une commande, c'est un fichier Markdown qui devient un /raccourci.**

◆ CRÉER UNE COMMANDE

Fichier .md
dans commands/

→

/nom
la déclenche

→

Sous-dossier
devient un préfixe

◆ PASSER DES ARGUMENTS

- **$ARGUMENTS**
  - Tout ce qui suit

- **$1, $2…**
  - Un argument par position

Un bloc !`commande` injecte une sortie shell — et s'exécute immédiatement.

Les serveurs MCP : connecter le monde

Tisser des ponts entre les îles de données.

MCP branche des outils externes, en local ou à distance.

◆ DEUX TYPES DE SERVEURS

- **Remote**
  - On donne une URL
  - Service en ligne
- **Local**
  - Une commande stdio
  - Sur ta machine

◆ AJOUTER PUIS UTILISER

opencode mcp add
déclare le serveur

→

mcp.servers
l'emplacement en config

→

serveur_outil
noms normalisés

Dans l'interface : /mcps. Les permissions peuvent bloquer un outil.

Bien prompter & le fichier AGENTS.md

L'incantation qui ordonne le chaos en dialogue.

Un bon prompt : contexte, objectif, contraintes, format.

◆ LES BRIQUES D'UN BON PROMPT

Contexte	d'où on part
Objectif	ce qu'on veut
Contraintes	ce qu'on refuse
Format attendu	ce qu'on reçoit

Et demande un plan avant d'agir : ça économise du temps et des tokens.

◆ DONNER LE CONTEXTE DU PROJET

/init
génère le fichier

→

AGENTS.md
lu automatiquement

→

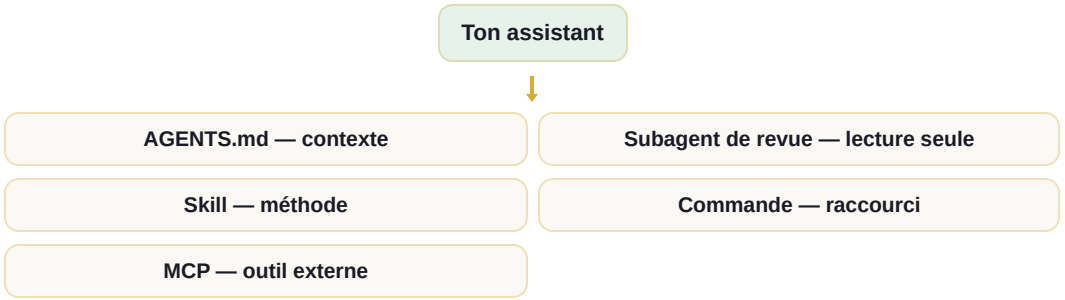
Règles courtes
mieux suivies

Mini-projet : ton assistant de A à Z

Élever, pierre à pierre, ta propre cathédrale de code.

> **Assemble tout : contexte, agent, skill, commande, MCP.**

◆ LES PIÈCES DE TON ASSISTANT



◆ CE QUE ÇA DÉMONTRE

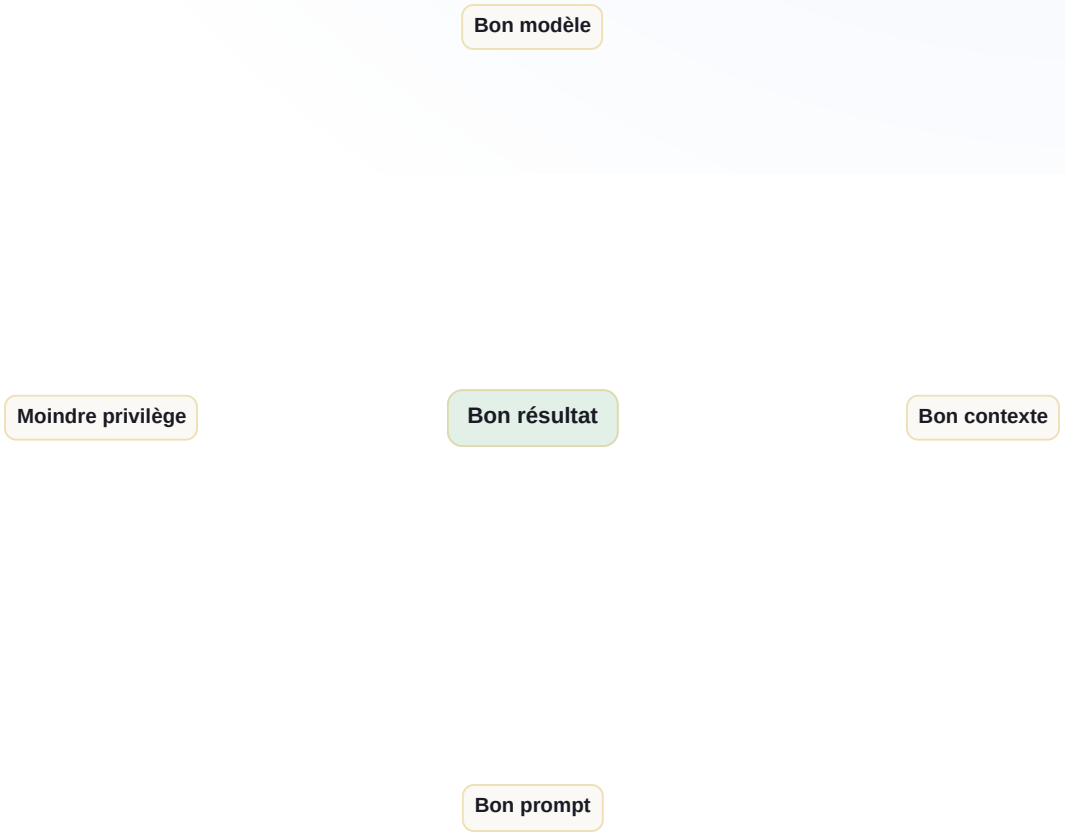
- **Moindre privilège**
  - Un agent = le minimum de droits
- **Versionné**
  - La config se partage avec l'équipe

Synthèse, dépannage & ressources

Le miroir universel interroge le soleil levant.

> **Bon modèle + bon contexte + bon prompt, et le moindre privilège.**

◆ LE TRIO GAGNANT



◆ LE PARCOURS COMPLET



Étends OpenCode seulement quand tu en as réellement besoin.

FIN DU PARCOURS

Tu sais maintenant où tu vas

1. L'IA prédit du texte, elle ne sait pas
2. Un provider branche les modèles
3. Gratuit = quotas serrés
4. Un agent = rôle + outils + droits
5. Skills, plugins, MCP : à la demande
6. Bon prompt, bon contexte, moins de coûts